



Acondicionamiento del NUCLEO-WL55JC al backend de Sigfox

Procedimiento de vinculacion end-to-end en la red 0G RC2

Guia tecnica de vinculacion

Flujo del ingeniero de hardware responsable

Como obtener PAC e ID del lado del chip,
registrar el device en el backend Sigfox
y enviar el primer uplink al 0G Network
desde la placa NUCLEO-WL55JC con firmware
configurado para Radio Configuration 2 (RC2 Mexico).

Plataforma: STMicroelectronics NUCLEO-WL55JC (MB1389)
Banda RF: 902.2 MHz uplink (RC2 Mexico, FCC)
Tecnologia: Sigfox 0G — Radio Configuration 2 (RC2)
Operador: 0G IoT Solutions (previamente WND Mexico)
Stack: STM32CubeWL V1.5.0
Documento base: AN5480 Rev 8 (marzo 2025)
Autor: Yahir Flores — Ingeniero de Hardware
Correo: yflores@iotnet.mx
Fecha: 29 de mayo de 2026
Version: v1.0

Sitio web del operador: <https://0giotsolutions.com/>

Resumen ejecutivo

Este documento describe el procedimiento completo para vincular una placa NUCLEO-WL55JC al backend de Sigfox bajo la configuracion regional RC2 (Mexico, 902.2 MHz uplink), incluyendo la extraccion de credenciales del chip, su validacion en el portal sfxp de STMicroelectronics, la escritura de las credenciales en la flash del MCU y la verificacion end-to-end con el primer uplink. El procedimiento se completo exitosamente el 29 de mayo de 2026 con los siguientes resultados verificables:

- Firmware `Sigfox_AT_Slave_CM4 V1.5.0` compilado y grabado en el MCU STM32WL55JCI.
- Chip certificate de 136 bytes extraido de `0x1FFF3F04` y validado por el portal `my.st.com/sfxp`.
- Credenciales reales emitidas por ST (ID, PAC, KEY AES-128) grabadas en `0x0803E500` (region `USER_Key_region_ROM`).
- Device registrado en `backend.sigfox.com` con Device ID `033E07FC`.
- Primer uplink exitoso recibido por la base station Sigfox 6BCE en 902.1782 MHz con RSSI `-127 dBm` (link quality *AVERAGE*).

El documento incluye los ocho pasos secuenciales del flujo, la lista de errores reales encontrados durante el procedimiento con sus soluciones, y los anexos tecnicos con direcciones de memoria, sintaxis AT compatible con V1.5.0, parametros RF normativos de RC2 Mexico y referencias oficiales.

Contents

Resumen ejecutivo	1
1 Contexto y objetivos	4
1.1 Objetivo	4
1.2 Plataforma	4
1.3 Resultados obtenidos	4
2 Prerequisitos	5
2.1 Hardware	5
2.2 Software	5
2.3 Cuentas y permisos	5
3 Roadmap secuencial de las 8 fases	6
4 Fase 1: Instalacion del entorno	7
4.1 STM32CubeIDE 2.0.0	7
4.2 STM32CubeProgrammer 2.6+	7
4.3 STM32CubeWL V1.5.0 — IMPORTANTE: descargar de st.com	7
4.4 Cliente serial	7
5 Fase 2: Conexion fisica y verificacion HW	8
5.1 Conexion mecanica	8
5.2 Verificacion en macOS	8
5.3 Connect en STM32CubeProgrammer	8
6 Fase 3: Compilar y flashear Sigfox_AT_Slave_CM4	10
6.1 Backup previo del flash (recomendado)	10
6.2 Mass erase	10
6.3 Importar el proyecto en CubeIDE	10
6.4 Region: runtime en V1.5.0 (no compile-time)	10
6.5 Build	10
6.6 Flashear via CubeProgrammer (recomendado)	10
6.7 Verificacion post-flash	11
7 Fase 4: Extraer el chip certificate	13
7.1 Concepto	13
7.2 Extraccion paso a paso	13
8 Fase 5: Portal sfxp y descarga de credenciales	14
8.1 Login y submission	14
8.2 Contenido del ZIP	14
8.3 Errores conocidos del portal	14

9 Fase 6: Escribir sigfox_data.bin en 0x0803E500	15
9.1 Por que esa direccion	15
9.2 Estructura del sigfox_data.bin (48 bytes)	15
9.3 Procedimiento	15
9.4 Verificacion	16
10 Fase 7: Validar y registrar en backend Sigfox	17
10.1 Acceso al backend	17
10.2 Caso especial: device pre-existente	17
10.3 Por que el PAC del backend difiere del PAC del .h	17
10.4 Si fuera un device nuevo (registro desde cero)	17
11 Fase 8: Primer uplink y verificacion end-to-end	19
11.1 Configurar region y potencia por AT	19
11.2 Enviar primer mensaje	19
11.3 Verificar en backend	19
11.4 Troubleshooting si no llega el mensaje	20
12 Errores encontrados durante el comisionamiento y soluciones	21
12.1 Error 12.1 — GitHub no incluye la app Sigfox	21
12.2 Error 12.2 — CubeIDE 2.0.0 pide ST-Link Server	21
12.3 Error 12.3 — Unable to get core ID	21
12.4 Error 12.6 — Region runtime, no compile-time	23
12.5 Error 12.7 — Confusion entre proyectos del workspace	23
12.6 Error 12.8 — PAC del backend distinto al PAC del .h	23
12.7 Error 12.9 — ZIP de 0 bytes del portal sfxp	23
12.8 Error 12.10 — Mass storage NODE_WL55JC no monta en macOS	24
12.9 Tabla resumen de errores	24
13 Anexos tecnicos	25
13.1 Direcciones de memoria criticas	25
13.2 Comandos AT compatibles V1.5.0	25
13.3 Parametros RF de Sigfox RC2 Mexico	25
14 Referencias y documentacion oficial	26
15 Resumen ejecutivo del flujo	27

1. Contexto y objetivos

1.1 Objetivo

Documentar y reproducir el procedimiento completo de vinculacion de una placa NUCLEO-WL55JC al backend de Sigfox bajo Radio Configuration 2 (RC2 Mexico), partiendo de un chip sin credenciales y terminando con un uplink visible en el backend. El alcance cubre desde la instalacion de herramientas hasta la validacion del primer mensaje recibido por la red 0G.

1.2 Plataforma

- Placa STMicroelectronics NUCLEO-WL55JC (board MB1389) con MCU STM32WL55JCI (Cortex-M4 + Cortex-M0+).
- Stack STM32CubeWL V1.5.0 con la aplicacion `Sigfox_AT_Slave_CM4` y la libreria propietaria `MW_SIGFOX` V1.8.0.
- Red Sigfox 0G bajo RC2 Mexico (902.2 MHz uplink, 905.2 MHz downlink, FCC), operada por 0G IoT Solutions (previamente WND Mexico).

1.3 Resultados obtenidos

Variable	Valor
Device ID Sigfox	033E07FC
PAC inicial (issued by ST)	B06EA49C9E0F11F4
PAC rotado post-registro	BC07485F3FFF8B71
Device Type	Nucleo_WL
Group	DHW
Contract	UnaTag_test (9 mensajes restantes tras registro previo)
Frecuencia uplink medida	902.1782 MHz
Base station receptora	6BCE
RSSI primer uplink	-127.00 dBm
Frames recibidos	1 de 3 (TX1/TX2/TX3)
Link Quality Indicator	AVERAGE (3/5 barras)
Payload transmitido	48656C6C6F ("Hello")
Delay backend	3.2 segundos

Table 1. Resultados del comisionamiento exitoso.

2. Prerequisitos

2.1 Hardware

- Placa NUCLEO-WL55JC (MB1389).
- Antena SMA macho compatible con banda 868/915 MHz (incluida en el kit estandar).
- Cable USB micro-B (longitud razonable; los muy largos a veces causan errores de enumeracion en macOS).
- Computadora con macOS, Windows o Linux (este documento detalla macOS).
- Cobertura Sigfox en el sitio de pruebas (verificar en <https://coverage.sigfox.com>).

2.2 Software

- STM32CubeIDE 2.0.0 o superior.
- STM32CubeProgrammer 2.6 o superior (**critico**: el boton de Sigfox Credentials se introdujo en 2.6).
- STM32CubeWL V1.5.0 *descargado desde st.com*, no desde GitHub (ver Capitulo 12, error 12.1).
- Cliente serial: Arduino IDE Serial Monitor, **screen**, o CoolTerm.
- Navegador con cookies habilitadas para `my.st.com` y `backend.sigfox.com`.

2.3 Cuentas y permisos

- Cuenta *myST* activa con acceso a sfxp.
- Cuenta admin en `backend.sigfox.com` con permisos sobre el grupo DHW (asignada por OG IoT Solutions).
- Token disponible en algun contrato del grupo (al menos 1 mensaje libre para validar).

3. Roadmap secuencial de las 8 fases

El flujo completo se organiza en ocho fases consecutivas, agrupadas en cuatro etapas logicas: setup y hardware, extraccion de credenciales, flasheo y backend, y validacion final.

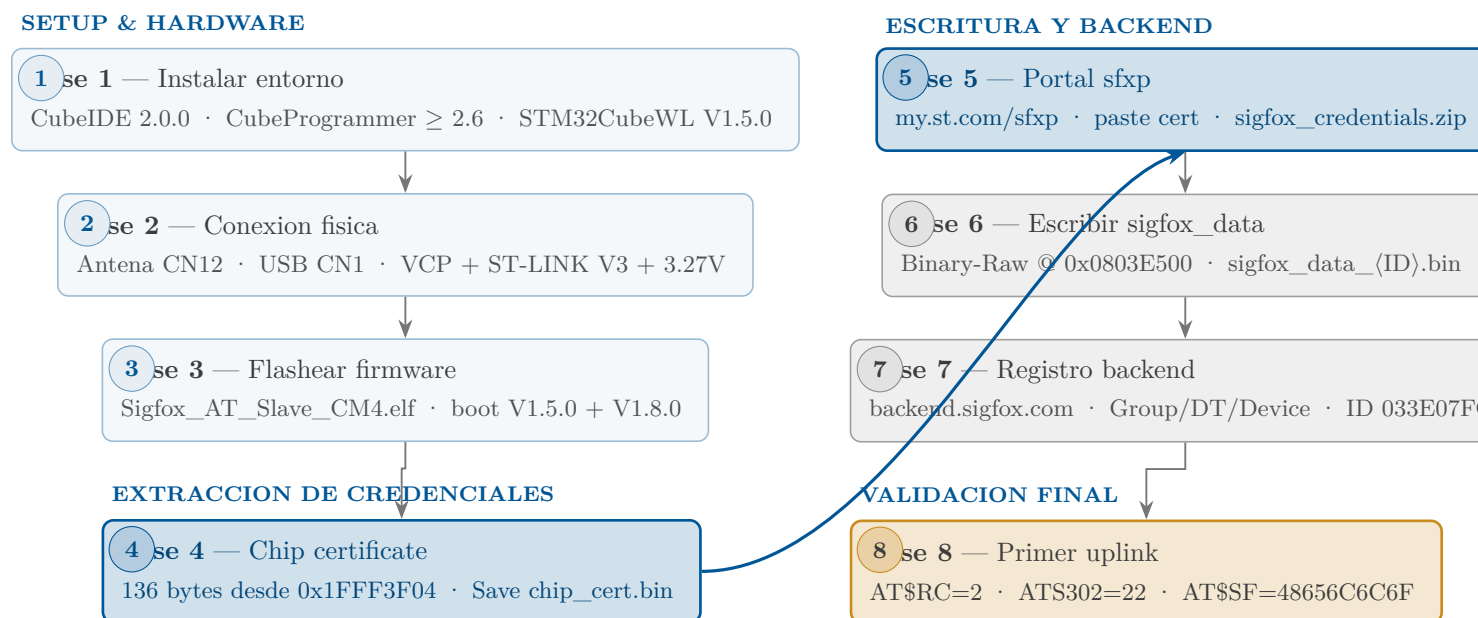


Figure 1. Mapa de las 8 fases del procedimiento de vinculacion del NUCLEO-WL55JC al backend Sigfox RC2. Azul claro indica setup, azul fuerte la extraccion de credenciales (etapa critica), gris la escritura y backend, ambar la validacion final.

Lista textual:

1. Instalar herramientas (CubeIDE, CubeProgrammer, CubeWL desde st.com, cliente serial).
2. Conectar la placa, verificar ST-LINK V3, VCP y target voltage.
3. Compilar y flashear **Sigfox_AT_Slave_CM4**; validar boot banner y AT por VCP.
4. Extraer chip certificate de 136 bytes con CubeProgrammer (0x1FFF3F04).
5. Subir certificate a my.st.com/sfxp y descargar sigfox_credentials.zip.
6. Escribir sigfox_data_{ID}.bin en 0x0803E500 via CubeProgrammer.
7. Validar AT\$ID y AT\$PAC reales; verificar registro en backend Sigfox.
8. Enviar primer uplink AT\$SF=48656C6C6F y validar recepcion en backend.

4. Fase 1: Instalacion del entorno

4.1 STM32CubeIDE 2.0.0

- Descarga: <https://www.st.com/en/development-tools/stm32cubeide.html>
- Requiere registro *myST*. Versiones macOS Intel/Apple Silicon disponibles.
- Instalar en ruta sin espacios. Drivers ST-LINK incluidos.

4.2 STM32CubeProgrammer 2.6+

Critico para nuestro flujo: la pestana *Sigfox Credentials* (que usaremos en Fases 4 y 6) solo existe en versiones ≥ 2.6 . La version instalada en este comisionamiento fue 2.22.

4.3 STM32CubeWL V1.5.0 — IMPORTANTE: descargar de st.com

No clonar desde GitHub. El repositorio publico <https://github.com/STMicroelectronics/STM32CubeWL> **no incluye** la aplicacion Sigfox por restricciones de licencia (stack propietario de UnaBiz). Si clonas con `git clone`, la carpeta `Applications/Sigfox/` no estara presente. Ver Capitulo 12, error 12.1.

Descarga oficial: <https://www.st.com/en/embedded-software/stm32cubewl.html>

Aceptar el license agreement (incluye terminos Sigfox/UnaBiz). Bajara un ZIP tipo `stm32cubewl-v1-5-0.zip` de aproximadamente 78 MB.

Descomprimir en ruta limpia:

```
mv ~/STM32CubeWL ~/STM32CubeWL_github_old # backup del clone incompleto
mkdir -p ~/ST
cd ~/Downloads
unzip stm32cubewl-v1-5-0.zip -d ~/ST/
```

Verificar que ahora *si* trae la app Sigfox:

```
find ~/ST -type d -name "Sigfox_AT_Slave" 2>/dev/null
```

Resultado esperado:

```
/Users/yflores/ST/STM32Cube_FW_WL_V1.5.0/Projects/B-WL5M-SUBG1/Applications/Sigfox/Sigfox_AT_Slave
/Users/yflores/ST/STM32Cube_FW_WL_V1.5.0/Projects/NUCLEO-WL55JC1/Applications/Sigfox/Sigfox_AT_Slave
```

Usaremos la ruta `NUCLEO-WL55JC1/...` (banda alta 865–928 MHz), que es la apropiada para Sigfox RC2 Mexico (902.2 MHz uplink).

4.4 Cliente serial

En macOS `screen` viene preinstalado. Alternativas: CoolTerm (GUI), Arduino IDE Serial Monitor. Configuracion obligatoria: **9600 baud, 8N1, sin flow control, fin de linea CR (Carriage return)**.

5. Fase 2: Conexion fisica y verificacion HW

5.1 Conexion mecanica

1. Atornillar la antena SMA al conector **CN12**.
2. Verificar jumpers (configuracion de fabrica):
 - JP1 (I_SoC): ON
 - JP3 (BOOT0): ON
 - JP4: posicion [1-2] (alimentacion desde ST-LINK USB)
 - JP7 (5V_PWR): ON
3. Conectar USB micro-B en **CN1** (lado ST-LINK).
4. Verificar que el **LED5 verde (PWR)** enciende.

5.2 Verificacion en macOS

```
ls /dev/cu.usbmodem*
ioreg -p IOUSB -l | grep -A 5 -i "stm32\|st-link\|stlink"
```

Esperado: enumera un puerto `/dev/cu.usbmodemNNNN` (el numero cambia entre reconexiones) y un dispositivo USB STLINK-V3 con serial number.

El *mass storage* NODE_WL55JC no siempre se monta en macOS Sonoma/Sequoia. **No es bloqueante** porque flasheamos via SWD, no por drag-and-drop.

5.3 Connect en STM32CubeProgrammer

Lanzar STM32CubeProgrammer. Configurar el panel ST-LINK:

- Port: **SWD**
- Frequency: 12000 kHz
- Mode: **Under reset** (no *Normal*)
- Reset mode: **Hardware reset**
- Speed: **Reliable**

Si dejas Mode: **Normal**, frecuentemente vas a ver el error **Unable to get core ID** o **No STM32 target found**. Esto pasa porque el firmware previo (o el de Sigfox que vamos a flashear) usa modos low-power agresivos que apagan las patas SWD. **Under reset** resetea el MCU justo antes de la conexion, lo que mantiene SWD activo en la ventana de reset.

Click **Connect**. Target information esperado:

Board: NUCLEO-WL55JC
Device: STM32WLxx
Device ID: 0x497
Revision ID: Rev Y
NVM size: 256 KB
CPU: Cortex-M4
Target voltage: 3.27 V

6. Fase 3: Compilar y flashear Sigfox_AT_Slave_CM4

6.1 Backup previo del flash (recomendado)

Si la placa se uso en proyectos anteriores, conviene respaldar el flash completo antes de sobrescribir:

- CubeProgrammer → Memory & File editing
- Address 0x08000000, Size 0x40000, Data width 32-bit
- Click **Read** → luego **Save As** con nombre descriptivo, ej. backup_placa_jc2_2026-05-29.hex.

6.2 Mass erase

CubeProgrammer → Erasing & Programming → **Full chip erase**.

6.3 Importar el proyecto en CubeIDE

- File → Open Projects from File System
- Directory: /Users/yflores/ST/STM32Cube_FW_WL_V1.5.0/Projects/NUCLEO-WL55JC1/Applications/Sigfox_AT_Slave_CM4
- Marcar el unico proyecto detectado → Finish.

6.4 Region: runtime en V1.5.0 (no compile-time)

Cambio importante respecto a versiones anteriores. En V1.4.0 y anteriores, la region se definia en `rfconf.h` con un `#define REGION_RC2`. **En V1.5.0 ese archivo no existe.** La region se selecciona en *runtime* via comando AT (`AT$RC=2`) y se persiste en EEPROM emulada. Ver Capitulo 12, error 12.5.

Por lo tanto: *no se edita codigo fuente antes de compilar*. Se compila la app tal cual viene.

6.5 Build

Click derecho sobre Sigfox_AT_Slave → **Clean Project** → **Build Project**. Resultado esperado:

```
arm-none-eabi-size Sigfox_AT_Slave.elf
  text    data     bss      dec       hex filename
 75512    312    10544    86368    15160 Sigfox_AT_Slave.elf

Build Finished. 0 errors, 0 warnings.
```

Footprint: 86 KB de 256 KB de flash. Sobra espacio.

6.6 Flashear via CubeProgrammer (recomendado)

Si intentas Run As → STM32 C/C++ Application desde CubeIDE 2.0.0, te pedira instalar *ST-Link Server* aparte. **No es necesario.** Es mas rapido flashear el `.elf` generado directamente con CubeProgrammer. Ver Capitulo 12, error 12.2.

Ubicar el .elf:

```
find /Users/yflores -name "Sigfox_AT_Slave.elf" 2>/dev/null
```

Esperado: /Users/yflores/ST/STM32Cube_FW_WL_V1.5.0/Projects/NUCLEO-WL55JC1/Applications/Sigfox/Si

En CubeProgrammer:

- Erasing & Programming
- File path: usar **Cmd+Shift+G** en el dialogo Browse para pegar la ruta completa
- Marcar: *Verify programming, Run after programming*
- No marcar *Skip flash erase*
- Click **Start Programming**

6.7 Verificacion post-flash

Tras el *Run*, aparecera el mensaje **Warning: Connection to device is lost. No es un error.** Es porque el firmware Sigfox entra inmediatamente en modo low-power tras boot y suelta SWD. Ver Capitulo 12, error 12.4.

Cerrar CubeProgrammer (**Cmd+Q**), presionar el boton B4 (NRST) en la placa para reset limpio, abrir Arduino Serial Monitor:

- Port: /dev/cu.usbmodemNNNN
- Baud: 9600
- Line ending: **Carriage return**

El firmware debe imprimir su boot banner:

```
APPLICATION_VERSION: V1.5.0
MW_SIGFOX_VERSION:   V1.8.0
MW_RADIO_VERSION:    V1.4.0
Attention command interface
SIGFOX APPLICATION READY
```

Test inicial:

```
AT
> OK

AT$ID
> FEDCBA98          (valor TEST por defecto, sin credenciales)
> OK

AT$PAC
> 0000000000000000 (valor TEST por defecto, sin credenciales)
> OK
```

Sintaxis AT en V1.5.0: usa el comando sin signo de pregunta (AT\$ID, no AT\$ID?). El ? en V1.5.0 invoca el texto de ayuda en lugar del valor. Ver Capitulo 12, error 12.5.

7. Fase 4: Extraer el chip certificate

7.1 Concepto

Cada STM32WL trae de fabrica un *chip certificate* de 136 bytes grabado en zona protegida (0x1FFF3F04 en la system memory) durante el wafer probe. Es unico por chip y solo ST puede validarlo criptograficamente.

7.2 Extraccion paso a paso

1. Cerrar Arduino Serial Monitor (libera VCP/ST-LINK).
2. Abrir CubeProgrammer → Mode **Under reset** → **Connect**.
3. En la barra lateral, click en el icono **Sigfox Credentials** (solo presente en CubeProgrammer ≥ 2.6).
4. Pestana **Read** → click boton **Read**.
5. Aparecen los 136 bytes en hex. Log esperado:

```
Size:      136 Bytes
Address: 0x1FFF3F04
Read progress:
Data read successfully
Time elapsed during the read operation is: 00:00:00.001
```

6. Click **Save chip certificate** → guardar como `chip_cert_<descripcion>.bin` (ej. `chip_cert_jc2_yflores_2026-05-29.bin`).
7. Click **Copy chip certificate** → queda en clipboard para pegarlo en el portal.

El chip certificate es **unico e irrepetible**. Si pierdes el `.bin`, tendras que volver a extraerlo cada vez. Etiqueta fisicamente la placa con el ID Sigfox una vez emitido para mantener trazabilidad chip-credenciales.

8. Fase 5: Portal sfxp y descarga de credenciales

8.1 Login y submission

URL: <https://my.st.com/sfxp>

1. Login con cuenta myST.
2. La pagina pide pegar el chip certificate. Hay dos opciones:
 - Pegar texto hex en el textarea (Cmd+V de lo copiado en CubeProgrammer).
 - Subir el archivo `.bin`.
3. Click el boton **Sigfox™ credentials**.
4. Si la validacion criptografica tiene exito, se descarga automaticamente `sigfox_credentials.zip`.

8.2 Contenido del ZIP

```
unzip -l sigfox_credentials.zip
Archive:  sigfox_credentials.zip
  Length      Date    Time    Name
-----
      48  05-29-2026  16:25  sigfox_data_033E07FC.bin
     1607  05-29-2026  16:25  sigfox_data_033E07FC.h
```

- `sigfox_data_<ID>.bin` — 48 bytes binarios. Es lo que escribiremos al flash en Fase 6.
- `sigfox_data_<ID>.h` — header C alternativo (no se usa en el flujo Binary-Raw).
- El sufijo del nombre *es el Device ID Sigfox*. En este caso: `033E07FC`.

8.3 Errores conocidos del portal

Si el ZIP sale de 0 bytes, hay *Service unavailable*, o no inicia la descarga, ver Capitulo 12 errores 12.6 y 12.7.

9. Fase 6: Escribir sigfox_data.bin en 0x0803E500

9.1 Por que esa direccion

En el linker script STM32WL55JCIX_FLASH.ld del proyecto Sigfox_AT_Slave, la variable USER_Key_region_ROM esta ubicada en 0x0803E500. Es una zona reservada al final de la flash que **no es tocada por el firmware** al programar el .elf. El binario sigfox_data.bin debe escribirse exactamente alli.

9.2 Estructura del sigfox_data.bin (48 bytes)

Inspeccionable con xxd:

```
xxd sigfox_data_033E07FC.bin
00000000: fc07 3e03 b06e a49c 9e0f 11f4 0100 0000  ..>..n.....
00000010: 51ad 0eba 0e92 e90b 7fa6 78f1 bf23 3e5d  Q.....x..#>]
00000020: 4d5f 3030 3438 5f41 4237 375f 3031 3fd5  M_0048_AB77_01?.
```

Descomposicion:

Offset	Tamano	Contenido
0x00 – 0x03	4 bytes	Device ID en little-endian (FC 07 3E 03 = 033E07FC)
0x04 – 0x0B	8 bytes	PAC (B0 6E A4 9C 9E 0F 11 F4)
0x0C – 0x0F	4 bytes	Flags / version (01 00 00 00)
0x10 – 0x1F	16 bytes	KEY AES-128 <i>cifrada</i> (no plaintext)
0x20 – 0x2D	14 bytes	ASCII identificador interno del chip (M_0048_AB77_01)
0x2E – 0x2F	2 bytes	Checksum / CRC

Table 2. Estructura del binario sigfox_data en V1.5.0.

9.3 Procedimiento

1. CubeProgrammer → Mode **Under reset** → **Connect**.
2. Click icono **Sigfox Credentials** en barra lateral.
3. Seccion inferior *SigFox credential provisioning*:
 - Configuration: **Binary-Raw**
 - Address: 0x0803E500 (autocompletado)
 - Binary file: click **Browse**, usar Cmd+Shift+G para pegar ruta a sigfox_data_033E07FC.bin
 - Header file: dejar vacio
4. Click **Write data**.
5. Resultado esperado: pop-up *SigFox credential provisioning succeeded*.

9.4 Verificacion

1. Disconnect en CubeProgrammer → Cmd+Q.
2. Reset fisico con B4.
3. Abrir Arduino Serial Monitor (9600, CR).
4. Probar:

```
AT
> OK

AT$ID
> 033E07FC    (ahora si el ID real, no FEDCBA98)
> OK

AT$PAC
> B06EA49C9E0F11F4    (PAC real, no ceros)
> OK
```

Si ambos valores coinciden con el contenido del .h, las credenciales estan correctamente cargadas y descifradas por el firmware.

10. Fase 7: Validar y registrar en backend Sigfox

10.1 Acceso al backend

URL: <https://backend.sigfox.com/auth/login>

Login con cuenta admin de OG IoT Solutions.

10.2 Caso especial: device pre-existente

En esta sesion, el device 033E07FC **ya estaba registrado** previamente por un colega (Jose Francisco Diaz Figueroa, 2025-10-22). Esto es frecuente con placas de laboratorio reutilizadas.

Al consultar el device en el backend se observo:

Campo	Valor observado
Identifier (hex)	33E07FC (cero a la izquierda omitido por display)
Name	WL55_A449 (nombre del registro anterior)
PAC mostrado	BC07485F3FFF8B71 (rotado, no coincide con el .h)
Device Type	Nucleo_WL
Group	DHW
Contract	UnaTag_test
Token validity	9 free messages left (1 sent previamente)
Last seen	2025-10-22 18:04:30
State	OK

Table 3. Estado del device al inicio de Fase 7.

10.3 Por que el PAC del backend difiere del PAC del .h

El PAC del .h (B06EA49C9E0F11F4) es el *PAC inicial* emitido por ST. Al usarlo para registrar el device por primera vez en el backend, el PAC se **consume y rota** a un nuevo valor (BC07485F3FFF8B71 en este caso). Este nuevo PAC es el que se usaria para *transferir* el device a otra cuenta o grupo en el futuro. **No es un error**, es comportamiento de seguridad estandar de Sigfox.

Lo que *si* importa para que el device transmita correctamente es la KEY AES-128, que no cambia y esta correctamente cargada en el MCU.

10.4 Si fuera un device nuevo (registro desde cero)

Para devices que aun no estan en el backend, los pasos serian:

1. Menu **Group** → usar el grupo existente o crear uno nuevo.
2. Menu **Device Type** → **New**:
 - Name: NUCLEO-WL55JC-RC2-DevTest
 - Group: el seleccionado arriba

- Keep alive period: 0 (sin alerta) o 1440 min (1/día)
- Payload display: Regular (raw)
- Downlink mode: Direct (permite 8 bytes de respuesta)
- Callbacks: vacío al inicio

3. Menu **Device** → **New**:

- Device ID: 033E07FC
- PAC: B06EA49C9E0F11F4 (*se consumirá*)
- Name: descriptivo (ej. Nucleo-WL55JC-001-Yflores)
- Device Type: el creado arriba
- Prototype: dejar sin marcar para producción real

11. Fase 8: Primer uplink y verificación end-to-end

11.1 Configurar región y potencia por AT

```
AT+RC=2
> OK
```

Configura la región a RC2 (México/USA/Canadá). Se persiste en EEPROM emulada y sobrevive resets.

Para fijar potencia TX en 22 dBm (máximo permitido por FCC en RC2):

```
ATS302=22
> OK
```

La sintaxis de set-power en V1.5.0 puede variar; si ATS302 no responde OK, dejar la potencia default. El firmware Sigfox por defecto usa el máximo permitido por la región configurada.

11.2 Enviar primer mensaje

Payload elegido: "Hello" = 48 65 6C 6C 6F en hex (5 bytes, dentro del límite de 12 bytes uplink).

```
AT+SF=48656C6C6F
> OK (después de ~6 segundos)
```

Lo que pasa internamente: la placa transmite 3 réplicas (TX1, TX2, TX3) en frecuencias de hopping distintas, cada una de aproximadamente 225 ms. El OK llega tras finalizar todas las transmisiones.

11.3 Verificar en backend

1. Navegar a backend.sigfox.com → Device → 33E07FC → **Messages**.
2. Refrescar (F5). El mensaje debería aparecer en 2–30 segundos.

Resultado obtenido en esta sesión:

Atributo	Valor
Time	2026-05-29 11:41:05
Delay	3.2 s
Seq Num	0
Data/Decoding	48656c6c6f
Base Station	6BCE
RSSI	-127.00 dBm
Frequency	902.1782 MHz
Frames	1 (de 3 transmitidas)
LQI	AVERAGE (3/5)
Location	gateway geolocalizado

Table 4. Mensaje recibido en backend Sigfox.

11.4 Troubleshooting si no llega el mensaje

Causa probable	Accion
Antena suelta o mal atornillada	Apretar firmemente en CN12
Region mal configurada	AT\$RC debe responder 2
Potencia baja	verificar $ATS302 \geq 20$
Sin cobertura RF	confirmar con 0G IoT Solutions el gateway mas cercano
ID/PAC del backend $\neq$ chip	comparar AT\$ID con backend
Token consumido	ver Token validity en Info del backend
Antena inadecuada para la banda	verificar que sea antena SMA de 868/915 MHz

Table 5. Tabla de diagnostico si el uplink no llega.

12. Errores encontrados durante el comisionamiento y soluciones

Esta seccion documenta los problemas *reales* encontrados en la sesion del 29 de mayo de 2026, en orden cronologico. Sirve como referencia para evitar repetir los mismos pasos en falso en futuras placas.

12.1 Error 12.1 — GitHub no incluye la app Sigfox

Sintoma:

```
git clone --recursive https://github.com/STMicroelectronics/STM32CubeWL.git
cd STM32CubeWL
ls Projects/NUCLEO-WL55JC/Applications/
FatFs  FreeRTOS  LICENSE.md  LoRaWAN  SubGHz_Phy
```

No aparece la carpeta Sigfox.

Causa: el stack de Sigfox es propietario de UnaBiz y ST no lo incluye en el repositorio publico de GitHub por restricciones de licencia. Solo se distribuye via el ZIP oficial de st.com despues de aceptar los terminos Sigfox.

Solucion: descargar el ZIP de <https://www.st.com/en/embedded-software/stm32cubewl.html>, aceptar la licencia, y descomprimir. Verificar:

```
find ~/ST -type d -name "Sigfox_AT_Slave"
```

12.2 Error 12.2 — CubeIDE 2.0.0 pide ST-Link Server

Sintoma: al hacer *Run As* → *STM32 C/C++ Application* aparece:

```
ST-LINK firmware verification
ST-Link Server is required to launch the debug session.
Please download it on www.st.com
```

Causa: CubeIDE 2.0.0 introdujo dependencia con ST-Link Server como componente separado para sesiones de debug.

Solucion: en lugar de instalar ST-Link Server, flashear el `.elf` directamente desde CubeProgrammer (que ya esta instalado y funciona). El *Erasing & Programming* acepta archivos `.elf` nativamente.

12.3 Error 12.3 — Unable to get core ID

Sintoma:

```
Error: Unable to get core ID
Error: No STM32 target found! If your product embeds Debug Authentication,
      please perform a discovery using Debug Authentication
\end{lstlisting>

\textbf{Causa:} el mensaje sobre Debug Authentication es \emph{falso positivo}
(STM32WL55JC no tiene TrustZone). La causa real es que el firmware previo
o el de Sigfox usa modos low-power agresivos que sueltan las patas SWD.
```

```
\textbf{Solucion:} cambiar \emph{Mode} de \texttt{Normal} a \textbf{Under
reset} y \emph{Reset mode} de Software a \textbf{Hardware reset}, luego
Connect.
```

Si aun falla, mantener presionado el boton B4 (NRST) mientras se hace Connect, soltarlo cuando empiece a buscar.

```
\subsection{Error 12.4 --- \texttt{Connection to device is lost} despues de Run}
```

```
\textbf{Sintoma:} tras un flasheo exitoso con \emph{Run after programming}:
```

```
\begin{lstlisting}
Download verified successfully
RUNNING Program ... Address: 0x08000000
Application is running, Please Hold on...
Start operation achieved successfully
Error: Unable to get core ID
Warning: Connection to device is lost
Disconnected from device.
\end{lstlisting}
```

```
\textbf{Causa:} \emph{esto no es un error.} El firmware Sigfox apenas
arranca entra en modo low-power (DeepSleep/STOP) entre transmisiones para
ahorrar bateria, y al hacerlo apaga las patas SWD. CubeProgrammer pierde
el debugger porque ya no hay con quien hablar via SWD.
```

```
\textbf{Solucion:} ignorar el warning. El firmware esta corriendo
correctamente. Para hablar con el ya hay que usar el VCP serial, no SWD.
```

```
\subsection{Error 12.5 --- Sintaxis AT cambio en V1.5.0}
```

```
\textbf{Sintoma:}
```

```
\begin{lstlisting}
AT$ID?
> AT$ID: Get the ID      (texto de ayuda, no el valor)
> OK
```

Causa: en V1.5.0, el ? al final de un comando AT invoca el *help text* en lugar de devolver el valor. En versiones anteriores, AT\$ID? devolvía el valor.

Solucion: usar el comando sin signo de pregunta:

```
AT$ID
> 033E07FC
> OK
```

Otros cambios de sintaxis observados:

- AT\$RC (sin parametro) puede no devolver valor; usar AT\$RC=2 para fijar y verificar via comportamiento.
- ATI puede no existir; el firmware imprime version solo en boot banner.

12.4 Error 12.6 — Region runtime, no compile-time

Sintoma: buscar `rfconf.h` con `Cmd+Shift+R` en CubeIDE no encuentra el archivo.

Causa: en V1.5.0, ST eliminó el archivo `rfconf.h` con defines tipo `REGION_RC2`. La region se selecciona en runtime mediante el comando `AT$RC=N` y se persiste en EEPROM emulada (en flash, leída al boot via `E2P_Read_ConfigWords`).

Confirmacion en el codigo: en `sgfx_app.c` de `Sigfox_AT_Slave`, la funcion `st_sigfox_open()` recibe `sfx_rc_enum_t sfx_rc` como parametro y hace switch sobre los casos `SFX_RC1`, `SFX_RC2`, `SFX_RC3A`, etc.

Solucion: no editar codigo fuente antes de compilar. Compilar tal cual y configurar region post-flash via AT.

12.5 Error 12.7 — Confusion entre proyectos del workspace

Sintoma: al hacer File Search por RC2 aparecen matches en dos proyectos:

- `LSM1x0A_SDK_LoRaWAN_Sigfox`: 5 matches en `sgfx_app.c`
- `Sigfox_AT_Slave`: 3 matches en `sgfx_app.c`

Causa: el workspace puede tener proyectos heredados de otras sesiones (`LSM1x0A` es para un modulo Murata distinto al NUCLEO).

Solucion: cerrar el proyecto irrelevante con *Close Project* y trabajar solo sobre `Sigfox_AT_Slave`. Verificar que el Build se hace sobre el proyecto correcto (click derecho sobre `Sigfox_AT_Slave` → Build, no `Cmd+B` con otro seleccionado).

12.6 Error 12.8 — PAC del backend distinto al PAC del .h

Sintoma: el backend muestra un PAC (`BC07485F3FFF8B71`) que no coincide con el PAC del header (`B06EA49C9E0F11F4`).

Causa: el PAC del header es el *PAC inicial* emitido por ST. Al usarlo para registrar el device por primera vez, se consume y rota a un PAC nuevo (usado para futuras transferencias). Este es comportamiento de seguridad estandar de Sigfox.

Solucion: no preocuparse. La KEY AES-128 (que es lo que autentica los uplinks) no cambia. El device transmite correctamente.

12.7 Error 12.9 — ZIP de 0 bytes del portal sfxp

Sintoma: la descarga del portal genera un ZIP vacio o no inicia.

Causa: el portal `my.st.com/sfxp` tiene historial de inestabilidad documentada en ST Community desde 2023.

Soluciones por orden:

1. Borrar cache del browser y reintentar.
2. Usar otro navegador (Chrome ↔ Firefox).

3. Reintentar en horarios distintos.
4. Confirmar que el certificate copiado son los 136 bytes completos, no un truncado.
5. Abrir ticket en ST Community categoria *STM32 MCUs Wireless*.
6. Para volumen >10 placas: solicitar *Sigfox Credential Request Agreement (CRA)* a UnaBiz / 0G IoT Solutions.

12.8 Error 12.10 — Mass storage NODE_WL55JC no monta en macOS

Sintoma: al conectar la placa por USB, el `ls /Volumes/` no muestra NODE_WL55JC (la unidad drag-and-drop).

Causa: macOS Sonoma/Sequoia tiene problemas conocidos con el mass storage virtual del ST-LINK V3 en cierta combinacion de firmware ST-LINK + version macOS.

Solucion: *no es bloqueante*. Como flasheamos por SWD via CubeProgrammer, el mass storage es prescindible. Verificar que al menos el VCP (`/dev/cu.usbmodemNNNN`) y el ST-LINK debug interface (visible en `ioreg`) sí están presentes.

12.9 Tabla resumen de errores

#	Sintoma	Solucion express
12.1	GitHub no trae Sigfox	Descargar ZIP de st.com (no clone)
12.2	CubeIDE pide ST-Link Server	Flashear .elf con CubeProgrammer
12.3	Unable to get core ID	Mode: Under reset + Hardware reset
12.4	Connection lost tras Run	Es esperado, firmware low-power
12.5	AT con ? muestra help	Usar AT sin signo de pregunta
12.6	rfconf.h no existe	Region es runtime en V1.5.0
12.7	Workspace confuso	Cerrar proyectos irrelevantes
12.8	PAC backend $\neq$ PAC .h	Normal, PAC rota tras registro
12.9	ZIP 0 bytes del portal sfxp	Cache, browser distinto, reintentar
12.10	Mass storage no monta	No bloqueante, flasheamos por SWD

Table 6. Errores reales encontrados durante el comisionamiento.

13. Anexos tecnicos

13.1 Direcciones de memoria criticas

Direccion	Tamano	Region	Contenido
0x08000000	256 KB	Flash (todo)	Firmware Sigfox_AT_Slave_CM4
0x0803E500	2 KB	USER_Key_region_ROM	sigfox_data.bin (credenciales)
0x1FFF3F04	136 B	System memory	Chip certificate (grabado en wafer probe)
0x20000000	64 KB	SRAM	RAM de trabajo

Table 7. Memory map relevante en STM32WL55JCI.

13.2 Comandos AT compatibles V1.5.0

Comando	Funcion
AT	Test de comunicacion (responde OK)
AT\$ID	Lee Device ID (4 bytes hex)
AT\$PAC	Lee PAC (8 bytes hex)
AT\$RC=N	Configura region (N=1..7), persiste en EEPROM
AT\$SF=<hex>	Envia frame uplink (hasta 12 bytes payload)
AT\$SF=<hex>,1	Envia frame con downlink request (consume 2 tokens)
AT\$SB=<bit>	Envia 1 bit (super eficiente para alarmas)
ATS302=N	Configura potencia TX en dBm (max 22 para RC2 FCC)
AT\$RST	Reset del MCU

Table 8. Comandos AT principales del firmware Sigfox_AT_Slave V1.5.0.

13.3 Parametros RF de Sigfox RC2 Mexico

Parametro	Valor
Frecuencia uplink central	902.2 MHz (frequency hopping FCC)
Frecuencia downlink central	905.2 MHz
Bandwidth total	192 kHz
Bandwidth por canal	600 Hz
Bit rate uplink	600 bps (D-BPSK)
Bit rate downlink	600 bps (GFSK)
Payload max uplink	12 bytes
Payload max downlink	8 bytes
Potencia maxima	+22 dBm (limitada por FCC)
Mensajes/dia uplink	140 (limite regulatorio FCC)
Mensajes/dia downlink	4 (limite Sigfox)
Listen Before Talk	No requerido (FCC, no ETSI)

Table 9. Parametros RF normativos de RC2 Mexico.

14. Referencias y documentacion oficial

Documento	URL
NUCLEO-WL55JC	https://www.st.com/en/evaluation-tools/nucleo-wl55jc.html
UM2592 (HW MB1389)	https://www.st.com/resource/en/user_manual/um2592-stm32wl-nucleo64-board-mb1389-stmicroelectronics.pdf
AN5480 (Sigfox + CubeWL)	https://www.st.com/resource/en/application_note/an5480-how-to-build-a-sigfox-application-with-stm32cubewl-stmicroelectronics.pdf
STM32CubeWL en st.com	https://www.st.com/en/embedded-software/stm32cubewl.html
GitHub STM32CubeWL (sin Sigfox)	https://github.com/STMicroelectronics/STM32CubeWL
Portal sfxp (credenciales)	https://my.st.com/sfxp
Backend Sigfox	https://backend.sigfox.com/
Sigfox API docs	https://support.sigfox.com/apidocs
Sigfox build (RC config)	https://build.sigfox.com/sigfox-radio-configurations-rc
Cobertura Sigfox	https://coverage.sigfox.com
Blog ST — Sigfox credentials	https://blog.st.com/stm32cubeprogrammer-sigfox/
ST Community Wireless	https://community.st.com/t5/stm32-mcus-wireless/bd-p/stm32-mcus-wireless-forum
0G IoT Solutions	https://0giotsolutions.com/

Table 10. URLs de referencia oficial.

15. Resumen ejecutivo del flujo

1. **Instalar:** STM32CubeIDE 2.0.0 + STM32CubeProgrammer ≥ 2.6 + STM32CubeWL V1.5.0 *desde st.com*.
2. **Conectar:** placa via CN1 USB. Verificar VCP y debugger enumerados. Connect en CubeProgrammer con *Mode: Under reset*.
3. **Flashear** Sigfox_AT_Slave_CM4.elf con CubeProgrammer. Validar boot banner V1.5.0 + V1.8.0 + V1.4.0 via Serial Monitor.
4. **Extraer** chip cert (136 bytes) desde CubeProgrammer → Sigfox Credentials → Read. Guardar como chip_cert.bin.
5. **Subir** a my.st.com/sfxp y descargar sigfox_credentials.zip con sigfox_data_<ID>.bin.
6. **Escribir** sigfox_data_<ID>.bin en 0x0803E500 via CubeProgrammer → Sigfox Credentials → Write data.
7. **Validar** con AT\$ID y AT\$PAC (sintaxis sin ?). Confirmar registro en backend Sigfox.
8. **Enviar** AT\$RC=2 → AT\$SF=48656C6C6F. Verificar mensaje en backend → Device → Messages en 2-30 s.

Documento generado el 29 de mayo de 2026

Acondicionamiento del NUCLEO-WL55JC al backend de Sigfox — v1.0

Yahir Flores — Ingeniero de Hardware — OG IoT Solutions

<https://Ogiotsolutions.com/>